



 770 Chapel Street
New Haven, CT 06510

 203-401-8768

 mydae.org

Project Design Documentation:

Last updated: July 29th, 2026

1. Project Title & Version Control

Order Sync and Failure Tracking Dashboard

This project will receive customer orders, check the information, save the order in a database, send it to another system, and track whether it was successful or failed.

Version Control

Version: DRAFT.01

Date: 08/03/2026

Change Log: Briefly note what changed in this version.

2. Project Summary (2-3 sentences)

What is your project, why does it matter, and what value is it intended to provide? Write a brief summary that someone unfamiliar with the project can understand.

The Order Sync and Failure Tracking Dashboard receives orders, checks the information, saves them, and sends them to another system. It also shows when an order fails, explains what went wrong, and allows the user to try again.

3. Problem Statement / Use Case

What real-world problem or need does this project address? Who is the intended user, and when or why would they use it?

Businesses often use more than one system to manage orders. Sometimes an order does not transfer correctly, and workers may not know where the problem happened.

This project is intended for employees or technical support staff who need to:

- See whether an order transferred successfully.
- Find failed orders.
- View the error.
- Retry the order after the problem is fixed.



770 Chapel Street
New Haven, CT 06510

203-401-8768

mydae.org

4. Goals and Objectives

List 2-3 clear and specific project goals. Describe what the completed project must accomplish by the end of the 12-week development period.

By the end of the 12 weeks, the project should:

1. Receive, check, and save customer orders.
2. Send orders to an outside test system using an API.
3. Track successful and failed orders and allow failed orders to be retried.

5. Key Features / Functions

List the main features or functions the project must include. Focus first on the essential features needed for a complete, usable minimum viable product (MVP).

The minimum working project will include:

- Create and receive a new order.
- Check that required order information is included.
- Save orders in a SQL database.
- Send orders to an outside test API.
- Mark orders as successful or failed.
- Save error messages and activity logs.
- View order status on a simple dashboard.
- Retry failed orders.

6. Tech Stack and Tools

List the languages, frameworks, platforms, databases, APIs, security tools, design tools, or other technologies you expect to use. Briefly explain what each major tool will support in the project.

- **C#:** Main programming language.
- **ASP.NET Core:** Used to build the backend and REST API.
- **SQL Server Express:** Free database for saving orders, statuses, and errors.
- **Entity Framework Core:** Connects the C# program to the database.
- **Swagger:** Tests the API without needing Postman.
- **Serilog:** Saves program activity and error logs.
- **Next.js:** Creates a small dashboard for viewing orders.
- **Git:** Tracks changes made to the code.
- **GitHub:** Stores and displays the project online.
- **VS Code:** Used to write and run the code.
- **Free mock API:** Acts like the outside system receiving the orders.

No real customer, payment, or private information will be used.



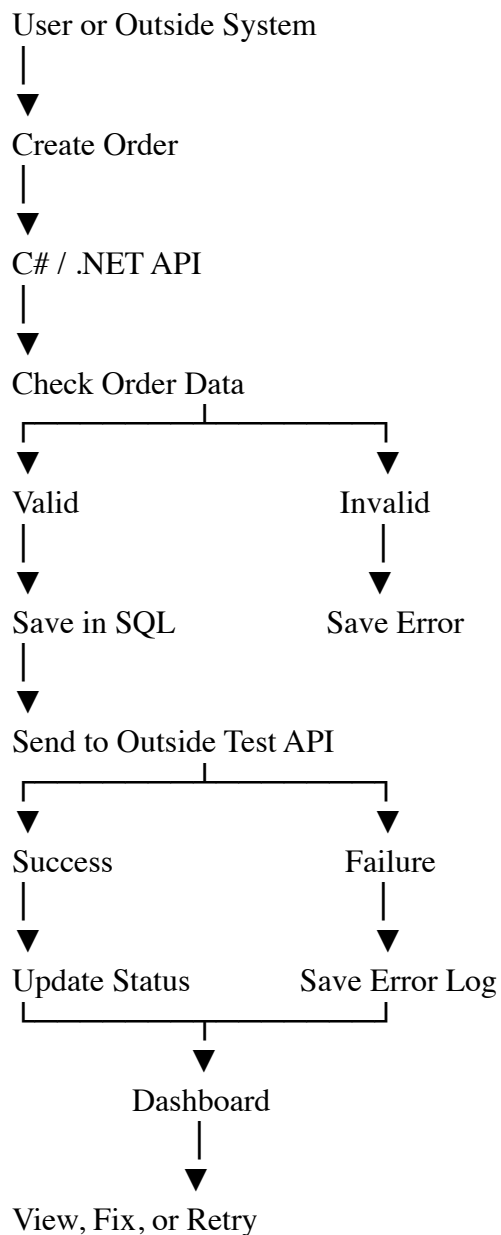
 770 Chapel Street
New Haven, CT 06510

 203-401-8768

 mydae.org

7. Architecture / Workflow Diagram

Create a simple diagram or flowchart showing how the project works from start to finish. Show the user or system input, the major steps or components, and the expected output. You may use AI to help draft the diagram, but review it and make sure it accurately reflects your project.





 770 Chapel Street
New Haven, CT 06510

 203-401-8768

 mydae.org

8. Timeline / Weekly Milestones

Use this section to plan how your project will move through the software development life cycle (SDLC) over 12 weeks. Adjust the specific tasks to fit your project, while keeping the overall progression from planning through release.

Week	Project Outcome	SDLC Stage and What It Means
Week 1	Project idea, title, purpose, intended user, and problem are clearly defined.	Discovery and Planning: Understand the problem before deciding what to build.
Week 2	Project requirements, scope, constraints, and essential MVP features are documented.	Requirements Analysis: Define what the project must do and what is outside the current scope.
Week 3	Success criteria, user stories or use cases, and a prioritized project backlog are completed.	Requirements and Project Planning: Turn the project idea into clear, testable work.
Week 4	Architecture, workflow, data needs, interface ideas, and technology choices are designed.	System Design: Decide how the parts of the solution will work together before full development begins.
Week 5	Development environment, repository, project structure, and an initial prototype or foundation are established.	Implementation Setup: Prepare the tools and build the first working structure of the project.
Week 6	The first group of core MVP features is built and reviewed against the requirements.	Implementation: Develop the essential functions in small, testable pieces.
Week 7	Remaining core features are built and connected into one working solution.	Implementation and Integration: Combine components and confirm that they work together.
Week 8	Core functions are tested; defects, missing requirements, and integration issues are documented.	Testing and Quality Assurance: Check whether each part behaves as intended and identify problems early.
Week 9	User, system, security, accessibility, or performance testing is completed as appropriate, and priority issues are fixed.	Validation and Refinement: Confirm that the solution works for its intended user and use case.
Week 10	The project is stabilized, code and content are cleaned up, and technical and user documentation are completed.	Release Preparation: Improve reliability, usability, and clarity before publishing.
Week 11	The project is deployed or published in its final environment, and the demonstration or presentation is prepared.	Deployment: Make the completed solution available and confirm it works outside the development environment.
Week 12	Final acceptance testing, project demonstration, documentation review, reflection, and future recommendations are completed.	Release and Maintenance Planning: Verify completion, communicate the result, and identify what should happen next.



 770 Chapel Street
New Haven, CT 06510

 203-401-8768

 mydae.org

9. Risks and Risk Mitigation

Identify risks that could affect successful completion of the project and explain how you will reduce or respond to each risk. Consider scope, time, technical difficulty, missing requirements, data, security, access to tools, and dependencies.

Learning C# may take longer than expected

Response: Keep the project small and learn only the C# needed for each feature.

The project may become too large

Response: Complete the backend, database, integration, and retry system before adding extra features.

The outside API may stop working

Response: Use a free mock API or create a second small API inside the project.

Database or connection problems

Response: Test the database connection early and save working versions in GitHub.

Not enough time for the dashboard

Response: Use Swagger to demonstrate the backend first. Add the Next.js dashboard only after the API works.

Security or private data concerns

Response: Use fake customer and order information only. Store passwords and connection settings outside the code.



 770 Chapel Street
New Haven, CT 06510

 203-401-8768

 mydae.org

10. Evaluation Criteria

How will you know the project is complete and working well? List at least 3 measurable success criteria, including whether the required features work, the project meets the intended user need, and the final product can be demonstrated or published.

The project will be considered complete when:

1. A new order can be received, checked, and saved in the database.
2. The order can be sent to an outside test API.
3. Successful and failed orders receive the correct status.
4. Failed orders show an error and can be retried.
5. The complete process can be demonstrated from beginning to end.
6. The project is published on GitHub with clear setup instructions.

11. Future Considerations

What improvements, additional features, maintenance, or next steps would you pursue after this version is complete? Separate future ideas from the work required for the current 12-week project.

After the 12-week version is complete, possible improvements include:

- User login and different employee roles.
- Email alerts for failed orders.
- More detailed reports and charts.
- Connecting to a real store or shipping API.
- Deploying the project online.
- Adding Docker.
- Adding automated tests.
- Supporting multiple outside systems.
- Adding Azure or another cloud service later.